THRIVE
Women for Women

Asynchronous JavaScript

THRIVE
Women for Women

Learning Objectives

- Understand synchronous vs asynchronous programming
- Understand callbacks, Promises, and async/await
- Fetch data from an API using fetch()
- Handle errors using .catch() and try/catch

Real goal:

👉 Write real async code that works in real applications.

Why Does This Matter?

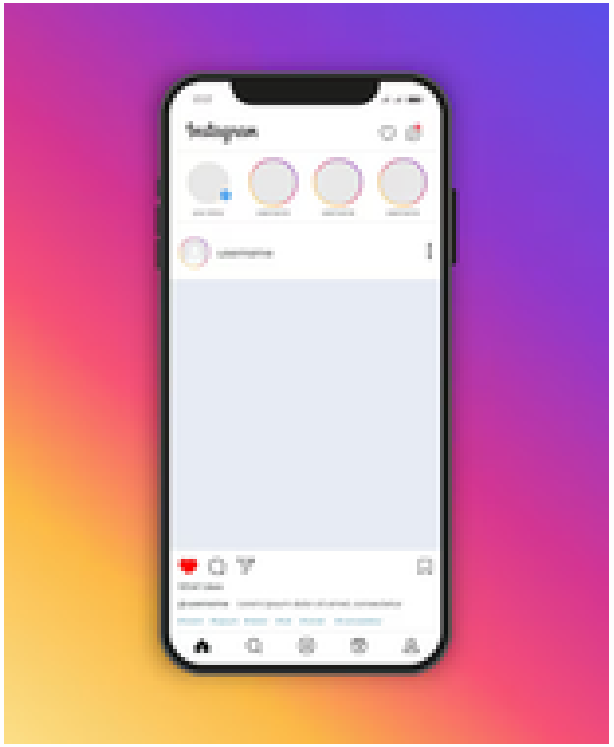
Where do we see async in real life?

- Loading Instagram/Twitter feeds
- Submitting login forms
- Searching on Google
- Loading products on Amazon

These all:

- Request data from a server
- Wait for a response
- Update the page when data arrives

Without async, the page would freeze 😬



Synchronous vs Asynchronous

Synchronous:

- One task at a time
- Waits until task finishes

Asynchronous:

- Does not block
- Allows other code to run

Synchronous Programming

- **Synchronous =** Code runs line by line, one after another.

JavaScript

```javascript
console.log("Start");
console.log("Cook food");
console.log("Eat food");
console.log("End");
```

Output:

Code

```
Start
Cook food
Eat food
End
```

The Problem with Sync

What if cooking takes 10 minutes?

```javascript
cookFood(); // takes 10 minutes
eatFood();
```

In synchronous code:

- Everything stops.
- Nothing else can run.

Restaurant analogy:

Imagine you must stand at the counter and wait 10 minutes before you're allowed to sit down.

Asynchronous Programming

Asynchronous = Start a task, but don't block everything while waiting.

JavaScript

```javascript
console.log("Order food");

setTimeout(() => {
  console.log("Food is ready!");
}, 2000);

console.log("Talk to friend");
```

Output:

Code

```
Order food
Talk to friend
Food is ready!
```

Used for:

- API calls
- Timers
- File reading

THRIVE
Women for Women

How do we handle code that finishes later (Async functions)?

What is a Callback?

A callback is a function passed into another function to run later.

```javascript
setTimeout(function() {
  console.log("Food is ready!");
}, 2000);
```

- We give setTimeout a function.
- That function runs after 2 seconds.

Callback Example

```javascript
function orderFood(callback) {
  console.log("Ordering food...");

  setTimeout(() => {
    console.log("Food ready!");
    callback();
  }, 2000);
}

orderFood(() => {
  console.log("Start eating!");
});
```

Flow:

- Order food
- Wait 2 seconds
- Callback runs

The Problem: Callback Hell 😬

```javascript
orderFood(() => {
  prepareTable(() => {
    getDrinks(() => {
      eatFood();
    });
  });
});
```

Problems:

- Hard to read
- Hard to debug
- Looks like a pyramid

Promises

A Promise is an object representing a value that will be available in the future.

Restaurant analogy:

- You get a receipt number.
- That receipt promises:
- “You will get food later.”

Promise States

A Promise has 3 states:

1. 🕐 Pending – still cooking
2. ✅ Fulfilled – food ready
3. ❌ Rejected – kitchen burned down

Using .then() and .catch()

```javascript
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```

What's happening?

- fetch() → returns a Promise
- .then() → runs when successful
- .catch() → runs if something fails

The Fetch API

What is fetch()?

- fetch() is a built-in JavaScript function.
- It is used to request data from a server.
- It returns a Promise.
- It works asynchronously (it does not block the page).

JavaScript

```javascript
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```

Async/Await (The Sweet Version)

Promises are powerful.

But chaining .then() can get messy.

Async/await makes it look synchronous.

Rules:

- async before function
- await inside async function only

You **CANNOT** use await without async.

Async/Await Example

```javascript
JavaScript

async function getPost() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts/1"
  );

  const data = await response.json();

  console.log(data);
}

getPost();
```

- Same logic.
- Cleaner.
- Easier to read.

Error Handling Problem

fetch does NOT throw error for 404!

```javascript
fetch("wrong-url")
```

It only fails if:

- Network fails
- Internet down

So we must check manually:

```javascript
if (!response.ok) {
  throw new Error("Request failed");
}
```

Try/Catch with Async/Await

JavaScript

```javascript
async function getPost() {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("API Error");
    }

    const data = await response.json();
    console.log(data);

  } catch (error) {
    console.log("Something went wrong:", error);
  }
}
```

Recap

Today we learned:

- Synchronous vs asynchronous
- Callbacks
- Callback hell
- Promises (pending, fulfilled, rejected)
- fetch()
- async/await
- try/catch error handling

Main idea:

JavaScript doesn't wait — it schedules and continues.

In-Class Exercise

Build a Mini Post Viewer

Use:

https://jsonplaceholder.typicode.com/posts

Tasks:

1. Fetch all posts
2. Display titles in console
3. Add error handling
4. Bonus: Fetch single post by ID

Home work

Create:

“Random User Viewer”

API:

https://randomuser.me/api/

Requirements:

- Fetch user
- Display name + email
- Handle errors
- Use async/await

Bonus:

- Add loading message

Final Mental Model

🍔 Restaurant Model Recap:

- Sync = stand in kitchen and wait
- Callback = “Call me when ready”
- Promise = receipt number
- Async/await = waiting politely at table
- try/catch = complain if order wrong

Question : What do you think is the output for this code and why?

```
console.log("first");

setTimeout(() => {
    console.log("working");

},0);

console.log("last");
```

Q/A